TASK - 9

(data is stored in log file and present data in state file)

TASK :

Two Esp32s were used each will generate 5 random telemetries such as temperature, lighting, moisture etc…. Data from both esps were sent to the laptop which acts as server using mqtt protocol. Here we store the present values as well as the previous value. The present value is stored inside state file and previous were stored in log file

1. The State File (state.json)

"State" refers to the exact current condition of the system.This file only holds the latest, single most recent reading from your ESP32s. Every single time a new reading arrives, the old reading in this file is completely erased and overwritten with the brand-new data.

2. The Log File (log.txt)

"Log" refers to a historical ledger or record book.This file holds the entire history of all readings from the past. When a new reading arrives, you append (add) it to the bottom of the file without erasing anything that came before it.

Workflow

Whenever ESP1 or ESP2 sends data over MQTT,Python script reads the data.
It looks at what is currently inside state.json and copies it over to log.txt (saving the "previous" state into history).It takes the brand-new data and overwrites state.json (updating the "present" state).

1. Create Project Folder

1. Create a folder on a laptop named IoT_Project.
2. Open this folder in VS Code (File -> Open Folder).
3. Create a new file inside it named receiver_with_storage.py.

2.Step 1: Program ESP32 Node 1 (Thonny)

Connect first ESP32 to Thonny. This script uses the urandom library to generate random numbers to simulate 5 telemetry parameters (Light, Battery, Temperature, Distance, and Moisture).
Create a new file in Thonny and save it as main.py on first ESP32:

```
import machine
```

```
import time
import network
import urandom # For generating random telemetry data
import json
from umqtt.simple import MQTTClient
# Wi-Fi & MQTT Configuration
WIFI_SSID = "YOUR_WIFI_NAME"
WIFI_PASSWORD = "YOUR_WIFI_PASSWORD"
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = b"group_project/esp1" # Topic for ESP 1
CLIENT_ID = "ESP32_Node_1"
# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)
print("ESP1: Connecting to Wi-Fi...")
while not wlan.isconnected():
time.sleep(1)
print("ESP1: Connected!")
# Connect to MQTT Broker
print("ESP1: Connecting to MQTT...")
try:
client = MQTTClient(CLIENT_ID, MQTT_BROKER, port=MQTT_PORT)
client.connect()
print("ESP1: Connected to Broker successfully!")
except Exception as e:
print("ESP1: Broker connection failed:", e)
# Main Loop: Generate and send telemetry every 3 seconds
while True:
try:
# Simulate 5 telemetries with realistic random ranges
temp = round(urandom.uniform(20.0, 40.0), 2) # 20°C to
40°C
dist = round(urandom.uniform(10.0, 200.0), 2) # 10cm to
200cm
moist = round(urandom.uniform(30.0, 90.0), 2) # 30% to 90%
light = round(urandom.uniform(0.0, 100.0), 2) # 0% to 100%
batt = round(urandom.uniform(0.0, 100.0), 2) # 0% to 100%
# Package everything beautifully into a single JSON
dictionary
telemetry_data = {
"temp": temp,
"dist": dist,
"moist": moist,
"light": light,
"batt": batt
}
```

```
# Convert dictionary to JSON string string
payload = json.dumps(telemetry_data)
print("ESP1 Sending:", payload)
# Publish payload
client.publish(MQTT_TOPIC, payload)
except Exception as e:
print("ESP1 Error:", e)
time.sleep(3)
```

Step 2: Program ESP32 Node 2 (Thonny)

plug in second ESP32 to Thonny, and flash this code.
Instead of removing the first esp32 we can create an another file and select the other port in
thonny (bottom right) where second esp32 is connected and in the new file we can paste this
code
Save – ctrl+shift+s
Two options will be shown
a)Micropython device
b) This computer
select micropython device and save it in .py format (eg: boot.py, main.py)

```
import machine
import time
import network
import urandom
import json
from umqtt.simple import MQTTClient
# Wi-Fi & MQTT Configuration
WIFI_SSID = "YOUR_WIFI_NAME"
WIFI_PASSWORD = "YOUR_WIFI_PASSWORD"
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = b"group_project/esp2" # Topic for ESP 2
CLIENT_ID = "ESP32_Node_2"
# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)
print("ESP2: Connecting to Wi-Fi...")
while not wlan.isconnected():
time.sleep(1)
print("ESP2: Connected!")
# Connect to MQTT Broker
print("ESP2: Connecting to MQTT...")
```

```
try:
client = MQTTClient(CLIENT_ID, MQTT_BROKER, port=MQTT_PORT)
client.connect()
print("ESP2: Connected to Broker successfully!")
except Exception as e:
print("ESP2: Broker connection failed:", e)
# Main Loop
while True:
try:
# Simulate 5 telemetries
temp = round(urandom.uniform(20.0, 40.0), 2)
dist = round(urandom.uniform(10.0, 200.0), 2)
moist = round(urandom.uniform(30.0, 90.0), 2)
light = round(urandom.uniform(0.0, 100.0), 2)
batt = round(urandom.uniform(0.0, 100.0), 2)
telemetry_data = {
"temp": temp,
"dist": dist,
"moist": moist,
"light": light,
"batt": batt
}
payload = json.dumps(telemetry_data)
print("ESP2 Sending:", payload)
client.publish(MQTT_TOPIC, payload)
except Exception as e:
print("ESP2 Error:", e)
time.sleep(3)
```

Code in VS code (Main Program Part)

This code automatically creates the folder and files if they don't exist, and manages the state vs log tracking.

```
import paho.mqtt.client as mqtt
import json
import os
from datetime import datetime

MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
TOPIC_ESP1 = "group_project/esp1"
TOPIC_ESP2 = "group_project/esp2"

BASE_DIR = "telemetry_data"
STATE_FILE = os.path.join(BASE_DIR, "state.json")
LOGS_FOLDER = os.path.join(BASE_DIR, "logs")
```

```
# Function to safely ensure directories exist at any given time
def ensure_directories_exist():
    if not os.path.exists(LOGS_FOLDER):
        os.makedirs(LOGS_FOLDER, exist_ok=True)
        print("[*] Automatically recreated missing logs folder.")

def save_nested_folder_log(esp_node, current_data):
    # CRITICAL FIX: Make sure the folders exist before doing anything
else
    ensure_directories_exist()

    now = datetime.now()
    timestamp_str = now.strftime("%Y-%m-%d %H:%M:%S")

    # 1. Generate the unique ID for the main timing folder
    time_id = f"log_{now.strftime('%Y%m%d_%H%M')}"

    # 2. Determine the folder name based on the ESP node
    node_folder_name = "ESP32_NODE_1" if esp_node == "esp1" else
"ESP32_NODE_2"

    # 3. Construct the full nested path
    target_folder_path = os.path.join(LOGS_FOLDER, time_id,
node_folder_name)

    # Automatically create these specific sub-folders if they were
deleted
    os.makedirs(target_folder_path, exist_ok=True)

    # 4. Prepare the clean parameters layout
    node_index = "1" if esp_node == "esp1" else "2"
    structured_telemetry = {
        "Timestamp": timestamp_str,
        f"Temperature_t{node_index}": f"{current_data.get('temp')}
°C",
        f"Distance_d{node_index}": f"{current_data.get('dist')} cm",
        f"Moisture_m{node_index}": f"{current_data.get('moist')} %",
        f"Light_l{node_index}": f"{current_data.get('light')} %",
        f"Battery_b{node_index}": f"{current_data.get('batt')} %"
    }

    # 5. Save the file inside its specific device folder
    data_filename = os.path.join(target_folder_path,
f"node{node_index}_telemetry.json")
    with open(data_filename, "w") as f:
        json.dump(structured_telemetry, f, indent=4)

    # --- STEP 6: SAFE STATE.JSON UPDATING ---
```

```
    state_data = {}
    # Use a try-except block so it doesn't crash if state.json is
deleted or empty
    if os.path.exists(STATE_FILE) and os.path.getsize(STATE_FILE) > 0:
        try:
            with open(STATE_FILE, "r") as f:
                state_data = json.load(f)
        except (json.JSONDecodeError, FileNotFoundError):
            pass # If it's corrupted or missing, we just start fresh
with an empty dictionary

    state_data[esp_node] = {
        "last_updated": timestamp_str,
        "parameters": current_data
    }

    with open(STATE_FILE, "w") as f:
        json.dump(state_data, f, indent=4)


def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("[*] Connected to HiveMQ Broker!")
        client.subscribe(TOPIC_ESP1)
        client.subscribe(TOPIC_ESP2)
        print("[*] Crash-Proof Logging Engine Active. Waiting for
data...\n")
    else:
        print("[-] Connection failed")

def on_message(client, userdata, msg):
    try:
        payload_text = msg.payload.decode("utf-8")
        data = json.loads(payload_text)
        node = "esp1" if msg.topic == TOPIC_ESP1 else "esp2"

        print(f"[*] New data packet from {node.upper()}. Checking and
updating directories...")
        save_nested_folder_log(node, data)

    except Exception as e:
        print(f"[-] Error: {e}")

# Initial check when starting up the script
ensure_directories_exist()

# Initialize MQTT Client
client = mqtt.Client()
```

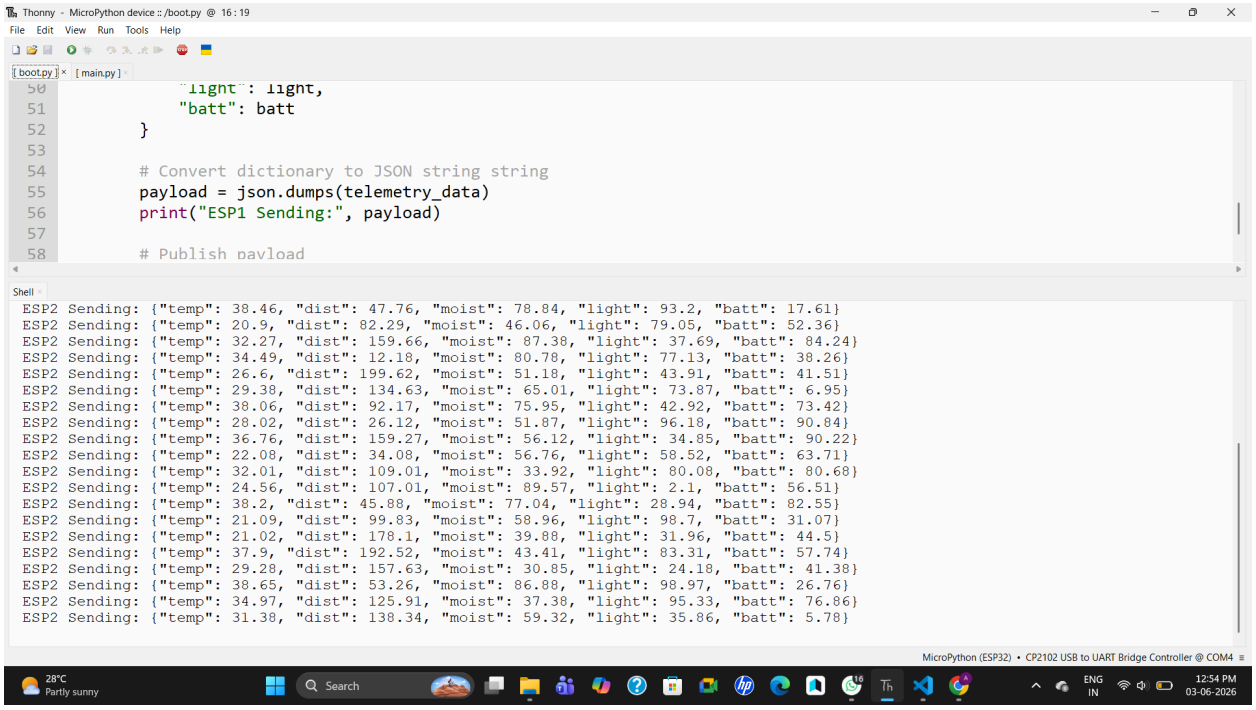
```
client.on_connect = on_connect
client.on_message = on_message
client.connect(MQTT_BROKER, MQTT_PORT, 60)

try:
    client.loop_forever()
except KeyboardInterrupt:
    client.disconnect()
```

Testing and Seeing the Files Change

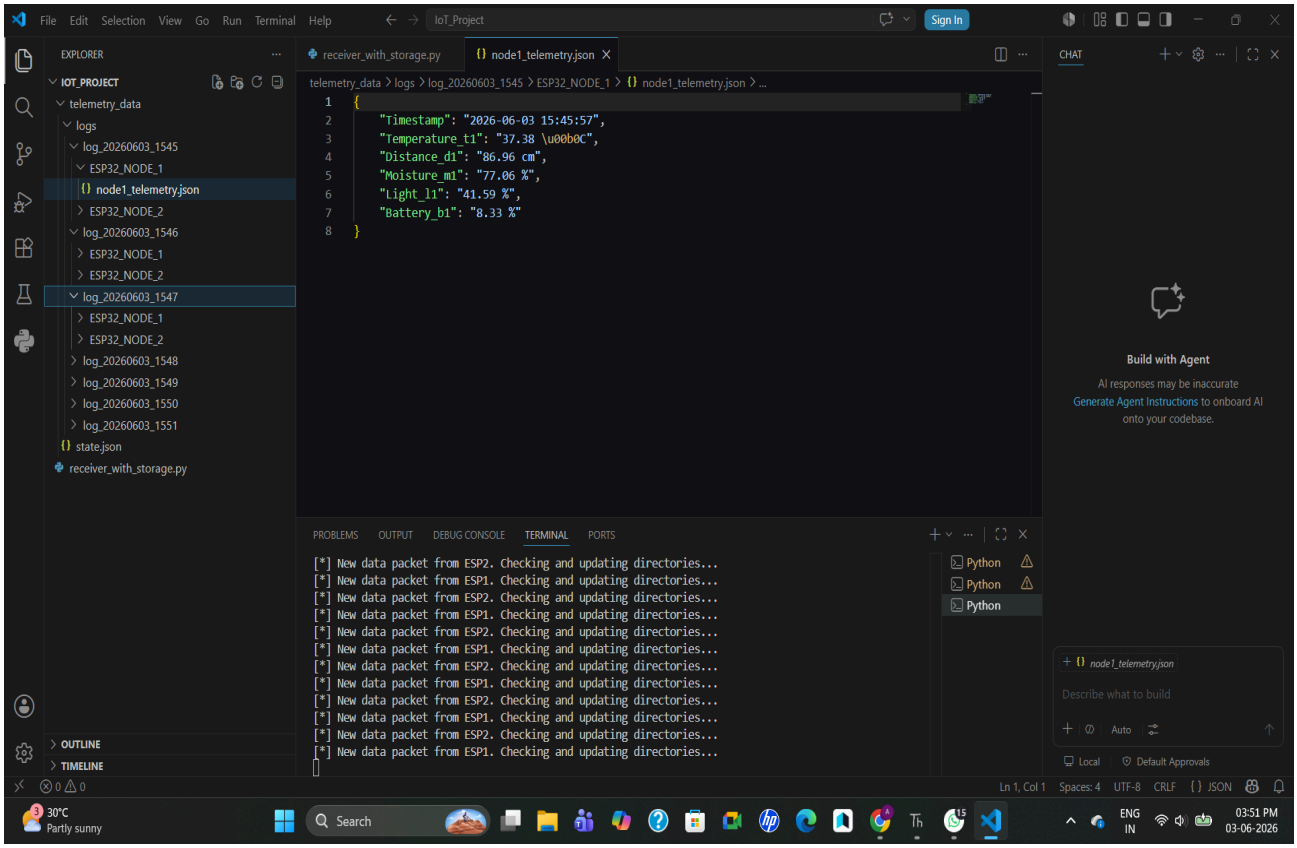
1. Run this updated receiver_with_storage.py file in VS Code.
2. Turn on your two ESP32 devices.
3. Look closely at your VS Code Left Sidebar Explorer window. You will notice a brand new folder automatically appear called **telemetry_data**.
4. Inside that folder, open **state.json**. you will notice the values constantly change every few seconds because it only keeps the current readings.
5. Next, open **log.txt**. You will see an ever-growing list tracking the entire history of timestamps showing both previous data entries and present data entries side-by-side.

OUTPUT

THONNY



VS CODE


File Edit Selection View Go Run Terminal Help
IoT_Project
Sign In
EXPLORER
IOT_PROJECT
telemetry_data
logs
log_20260603_1545
ESP32_NODE_1
node1_telemetry.json
ESP32_NODE_2
log_20260603_1546
ESP32_NODE_1
ESP32_NODE_2
log_20260603_1547
ESP32_NODE_1
ESP32_NODE_2
log_20260603_1548
log_20260603_1549
log_20260603_1550
log_20260603_1551
state.json
receiver_with_storage.py
OUTLINE
TIMELINE
receiver_with_storage.py
node1_telemetry.json
telemetry_data > logs > log_20260603_1545 > ESP32_NODE_1 > node1_telemetry.json > ...
{
"Timestamp": "2026-06-03 15:45:57",
"Temperature_t1": "37.38 \u00b0C",
"Distance_d1": "86.96 cm",
"Moisture_m1": "77.06 %",
"Light_l1": "41.59 %",
"Battery_b1": "8.33 %"
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[*] New data packet from ESP2. Checking and updating directories...
[*] New data packet from ESP1. Checking and updating directories...
[*] New data packet from ESP2. Checking and updating directories...
[*] New data packet from ESP1. Checking and updating directories...
[*] New data packet from ESP2. Checking and updating directories...
[*] New data packet from ESP1. Checking and updating directories...
[*] New data packet from ESP2. Checking and updating directories...
[*] New data packet from ESP1. Checking and updating directories...
[*] New data packet from ESP2. Checking and updating directories...
[*] New data packet from ESP1. Checking and updating directories...
[*] New data packet from ESP2. Checking and updating directories...
[*] New data packet from ESP1. Checking and updating directories...
Python
Python
Python
CHAT
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
node1_telemetry.json
Describe what to build
Auto
Local
Default Approvals
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
JSON
30°C
Partly sunny
Search
ENG
IN
03:51 PM
03-06-2026